20th Conference on Neurosymbolic Learning and Reasoning (NeSy 2026)

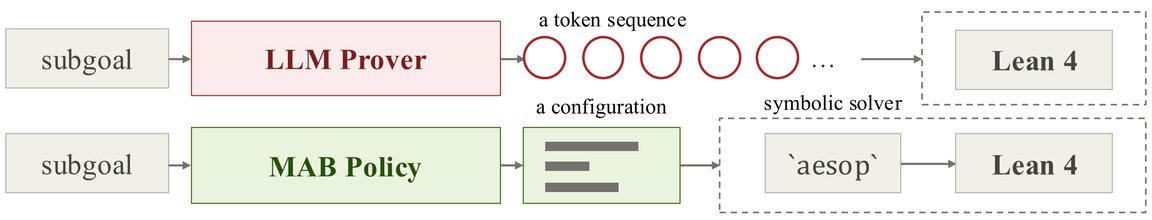
Steering Aesop for Theorem Proving: Learning to Configure Symbolic Solvers via Multi-Source Neurosymbolic Feedback

Kai-Yuan Jeng **Ru-Jun Wang** **Wen-Tsuen Chen**

Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan

Introduction & Motivation

- Neural theorem provers write formal proofs verified by Lean 4. Draft-Sketch-Prove (DSP) decomposes a theorem into *subgoals*, each proved by an LLM step prover or a symbolic solver such as `aesop`.
- LLM provers pursue Pass@k through token generation, yet the solver configuration remains **left at defaults or expert-tuned**. Such tuning is effective but laborious.
- We reframe subgoal proving as **solver configuration**, a **shift** from token sequences to configurations, and a fast path ahead of any LLM.



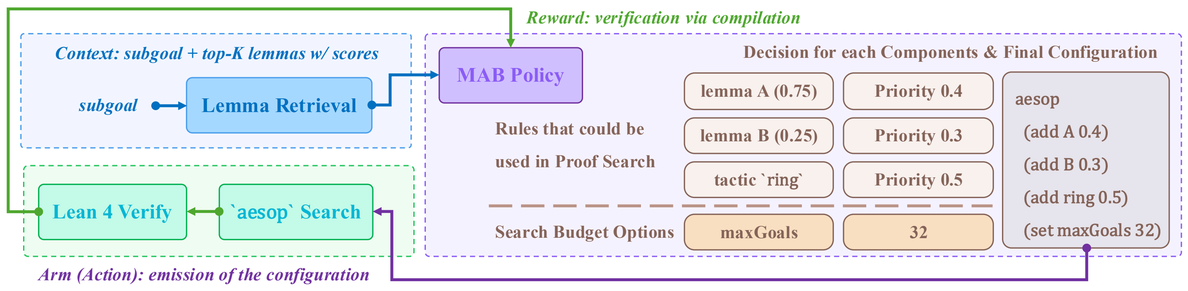
One decision configures the whole search, while an LLM samples for each token.

A miss by the solver costs negligible runtime, whereas a hit saves everything downstream.

Problem Formulation

Solver configuration as a contextual multi-armed bandit (MAB) problem

- **Context** $C \in \mathcal{C}$**:** the subgoal, its top-K retrieved lemmas, and their similarity scores.
- **Arm** $A \in \mathcal{A}$**:** one `aesop` configuration, i.e., rule registrations and search budgets. It factorizes into H components (M options each), so $|\mathcal{A}| = M^H$, combinatorially large.
- **Reward** $\mathcal{R} : \mathcal{C} \times \mathcal{A} \to \{+1, -1\}$**:** whether the configured `aesop` proves the subgoal, which is binary and deterministic given (C, A).



One configured `aesop` execution inside Lean 4 returns the deterministic reward.

Three challenges for training a contextual MAB policy under sparse binary reward

- **(C1) Reward sparsity:** reaching ε-suboptimality needs $\Omega(1/p_s)$ verifications, with p_s the success probability, and each verification takes seconds.
- **(C2) Unattributable failures:** a configuration has many components, so a randomly sampled failure cannot be directly attributed to any single component.
- **(C3) Demonstrator ceiling:** cloning verified successful configurations only reproduces them. Under sparsity, policy gradient reduces to cloning the policy's own successes.

Methodology: SATP

- **Dense supervision (C1):** **verified configurations** are **cached** and **reused**.
- **Controlled perturbation (C2):** flipping **one component of a success** yields a failing counterpart. We then construct preference pairs for DPO loss ($\mathcal{L}_{\text{dense}}$).
- **Exploration (C3):** REINFORCE with a self-critic greedy baseline ($\mathcal{J}_{\text{sparse}}$). **Thompson sampling via dropout** supplies newly verified successes beyond the ceiling.
- **Calibration constraint:** by penalizing only the joint **failure–overconfidence (low entropy)** event ($\mathcal{M}$), we keep the policy uncertain where it fails.

$$\min_{\theta} \Big(\mathcal{J}(\theta) := \underbrace{-\mathcal{J}_{\text{sparse}}(\theta)}_{\text{Exploration}} + \underbrace{\mathcal{L}_{\text{dense}}(\theta)}_{\text{Exploitation}} \Big) \quad \text{subject to} \quad \underbrace{\mathcal{M}(\theta) \le \gamma^{\text{calib}}}_{\text{Calibration}}$$

One training step (Algorithm 1)

REINFORCE update → **Perturb one component** → **DPO on preference pairs** → **Dual ascent on λ**

Feasibility of SATP (RQ1)

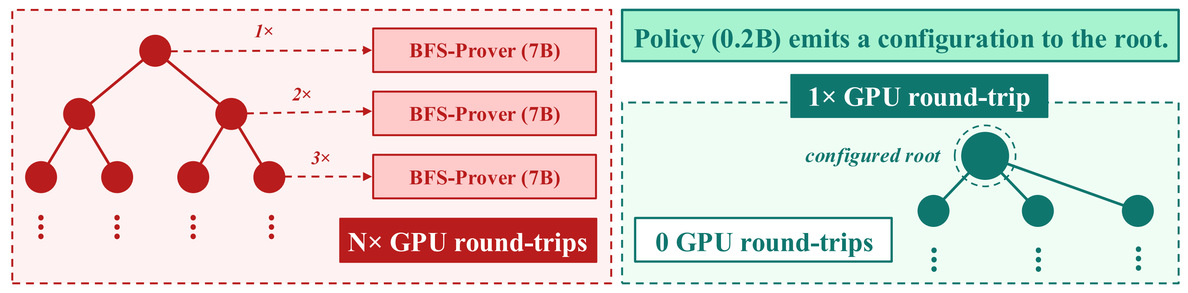
`satp` reaches a solve rate of **32.6%** $\pm$ **0.3%** on `miniF2F-test` across retraining runs, surpassing the demonstrator ceiling (29.1%). Training costs $\approx$ **50 RTX 5090 GPU-hours**, orders of magnitude less compute than LLM provers.

Method	Solve Rate
`aesop` (default)	10.7%
w/ tactic registration, no priority	13.5%
w/ behavioral cloning on seeded buffer	29.1%
`hammer`	26.6%
`ReProver` (Yang et al., 2023)	26.5%
`satp` (ours)	**32.6% $\pm$ 0.3%**
w/o $\mathcal{L}_{\text{dense}}$	12.3%
`satp_static` (distilled from `satp`)	32.4%
Expert-tuned `aesop`† (Cao et al., 2025)	35.2%

† *configuration outside $\mathcal{A}$, cf. `satp-v2`, 37.7%*

Composability of `satp` (RQ2)

We integrate `satp` into DSP, using `Qwen3.5-27B-FP8` as drafter and sketcher, evaluating on `miniF2F-test`. The cascade `satp` $\to$ `bfsaesop` orders provers by inference cost: `satp` configures the search once at the root (**one 0.2B forward pass**), while `bfsaesop` queries a 7B step prover at every node of the `aesop` search.

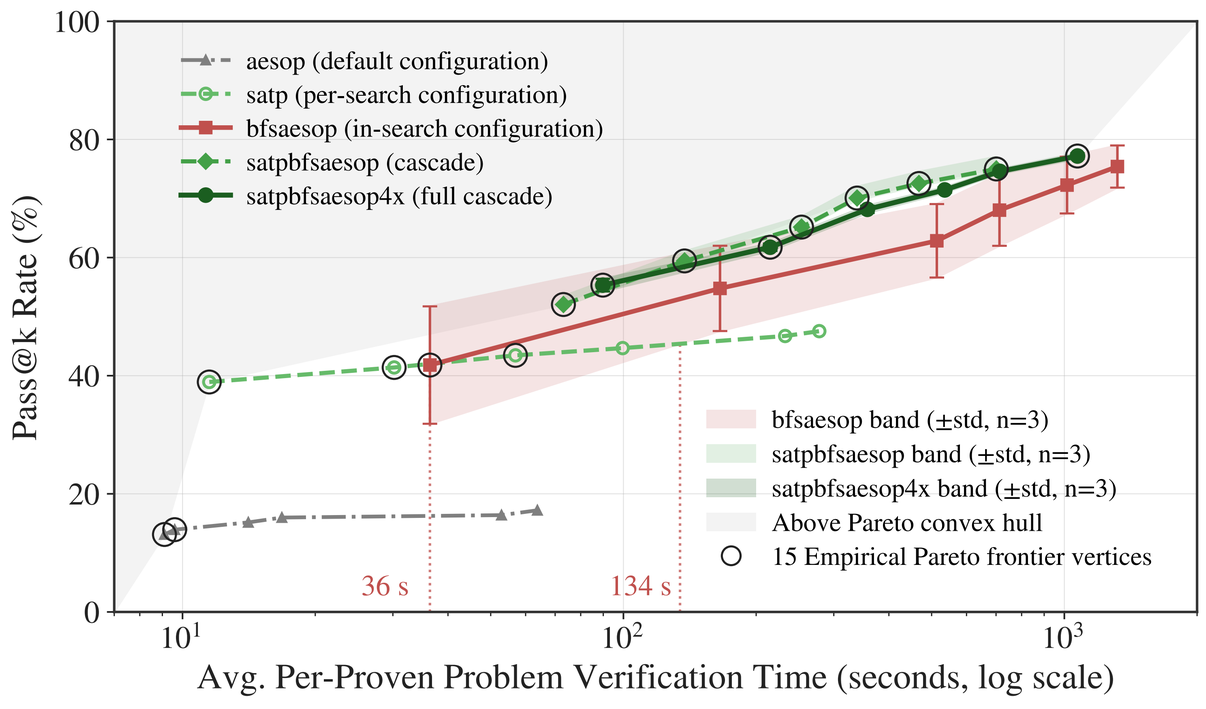


Per-node token generation (left, red) vs. pre-search solver configuration (right, green).

55.3% $\pm$ 1.1%
Pass@1 vs. `bfsaesop` baseline's 41.8%$\pm$9.9%

77.2% $\pm$ 0.5%
Pass@32 vs. Cao et al. (2025)'s reported 71.3%

82.6% $\pm$ 0.6%
of all solved subgoals proved deterministically by `satp`



Pareto frontier on `miniF2F-test` ($k \in \{1, 2, \ldots, 32\}$). Below 36 s only `satp` operates.

`satpbfsaesop` dominates `bfsaesop` at larger k with a tighter $\pm$std band.

Conclusion

- A **learnable, deterministic fast path** ahead of any LLM: via solver configuration, `satp` lets a complex LLM-based proving system prove theorems **with great confidence**.
- `satp`-v2 (Appendix I) reaches **37.7%** from an empty buffer $B_0 = \varnothing$, above every symbolic baseline, including the expert-tuned `aesop`.

`satp` tactic